

Logminer: Auditable Family-Aware Routing for Heterogeneous Log Anomaly Detection

André Olivier Marhegane Bitati*, Gédéon Nkishi Djenga*, Jean-Pierre Mugisha Muhire*
Josué Lungi Selemani*, Christian Kutabuna Kubakisa*, Jeff Mugaruka Chihyoka*, Witesyavwirwa Vianney Kambale†, Kyar

* Faculty of Information and Communication Technology, Tshwane University of Technology (TUT), Pretoria, South A

† Faculty of Engineering (Polytechnic), Department of Electrical and Computer Engineering, University of Kinshasa, Kinshasa.

‡ Institute for Smart Systems Technologies, University of Klagenfurt, Klagenfurt, Austria.

Corresponding author: André Olivier Marhegane Bitati

Emails: aolivier.marhegane@unikin.ac.cd, gedeon.nkishi@unikin.ac.cd, tomsmugisha@unikin.ac.cd

josue.lungi@unikin.ac.cd, christian.kutabuna@unikin.ac.cd, Mugarukachihyoka@gmail.com, kambalevw@tut.ac.za, kyandoghre.ky

Abstract—Security monitoring environments generate heterogeneous logs whose formats, semantics and statistical distributions differ across operating systems, network flows, SIEM exports and distributed systems. Applying a single monolithic detector to all sources may increase confusion between log families and reduce the interpretability of alerts. This paper presents Logminer, an auditable family-aware routing architecture for anomaly detection in heterogeneous system and network logs.

The prototype decomposes the workflow into specialized cooperating modules for collection, parsing, normalization, family-aware routing, anomaly detection, correlation, dashboard visualization and audit. The contribution is not a new standalone detector, but an auditable specialization layer: events are normalized into a common schema, assigned to a log family with explicit evidence and confidence, processed by a compatible lightweight model, then unified as anomaly candidates before being correlated into analyst-readable incidents.

Experiments cover Windows logs, Wazuh exports, Linux/auth logs, HDFS, BGL, CICIDS2017 and UNSW/CIC-DDoS data. The evaluation includes a controlled common-space ablation and an operational specialization ablation. The results support the feasibility of a local lightweight implementation, while showing that routing should be interpreted as controlled specialization rather than a universal F1 improvement on every dataset.

The prototype is positioned as an analytical complement to mature log management and HIDS/SIEM tools rather than as a replacement for production SOC platforms.

Index Terms—anomaly detection, log analysis, family-aware routing, multi-model architecture, cybersecurity, lightweight machine learning, SIEM complement, log correlation

I. INTRODUCTION

System and network logs are central to security monitoring, incident investigation and operational audit. However, modern infrastructures generate logs from heterogeneous sources: Windows event logs, Linux authentication logs, SIEM/HIDS exports, network-flow datasets, cloud logs and distributed-system traces. These sources differ not only in syntax, but also in event semantics, feature availability, temporal density and class imbalance. This heterogeneity complicates the design of a single anomaly detection pipeline.

Classical intrusion detection and log management research has long emphasized audit trails, deviation from normal behavior and the difficulty of reducing false positives [1], [2], [4].

More recent machine learning approaches provide promising results, but remain sensitive to dataset shift, closed-world assumptions, class imbalance and operational interpretability [3], [5], [6]. In practice, operational tools such as fail2ban, OSSEC, Wazuh or ELK-like stacks provide valuable collection, rule-based detection, indexing and visualization, but they do not by themselves solve the problem of controlled model specialization across heterogeneous log families.

This paper introduces Logminer, a lightweight prototype designed to evaluate a family-aware multi-model strategy for heterogeneous log anomaly detection. The term agent is used here in a software architectural sense: Logminer is composed of specialized cooperating modules with distinct functional roles, message exchange and workflow coordination. The prototype does not claim to implement fully autonomous cognitive agents, FIPA-style negotiation, reinforcement-learning cooperation or LLM-agent autonomy. It includes a bounded supervisor agent that performs a perception-state-decision-action cycle for local workflow adaptation, but this supervisor is treated as an implementation control layer rather than as the main scientific contribution.

The central research question is: when and why can auditable family-aware routing improve heterogeneous log anomaly detection compared with monolithic alternatives? The answer is not expected to be universal. Routing is evaluated as a mechanism that enables compatible feature spaces, explicit fallback behavior, interpretable routing reasons and family-specific operational policies.

The contribution of this paper is fourfold:

- 1) An auditable family-aware software-agent architecture for log analysis, including collector, parser, normalizer, router, detector, correlator, dashboard, audit and bounded supervisor roles.
- 2) A formal routing and fallback mechanism in which normalized logs are assigned to a log family through explicit evidence, confidence scoring and route explanations before being processed by a compatible detector.
- 3) A multi-source evaluation covering system logs, SIEM exports and network datasets, including controlled and

operational ablations, false positives, quasi-real-time workflow latency, resource consumption, supervisor stability and operational limits.

- 4) A cautious empirical positioning showing that family-aware routing is controlled specialization rather than a guaranteed F1 improvement on every dataset.

The paper deliberately uses cautious claims. Non-supervised outputs are treated as anomaly candidates, not confirmed intrusions. The validated distribution is mainly logical and modular, while large-scale multi-machine distribution remains future work. The remainder of the paper is organized as follows: Section II reviews related work, Sections III and IV present the methodology and routing mechanism, Section V details the implementation, Section VI describes the experimental setup, Section VII reports and discusses the results, Section VIII states the operational positioning and validity limits, and Section IX concludes the paper.

II. RELATED WORK

A. Anomaly Detection and Intrusion Detection

Denning's intrusion detection model introduced the idea of detecting deviations from expected user and system behavior through audit records [1]. Axelsson later structured IDS concepts and taxonomies, while Chandola et al. provided a broader survey of anomaly detection methods across point, contextual and collective anomalies [2], [4]. These foundations are directly relevant to log anomaly detection, where isolated rare events and correlated sequences may indicate abnormal behavior.

Machine learning has been widely investigated for cyber intrusion detection. Buczak and Guven review data mining and machine learning methods for cybersecurity, while Khraisat et al. discuss IDS datasets, challenges and evaluation issues [5], [6]. Sommer and Paxson warn against overinterpreting machine learning results in intrusion detection, especially when training and testing assumptions do not reflect operational environments [3]. This warning motivates the cautious positioning adopted in this work.

B. Log Parsing and Log-Based Detection

Before machine learning can be applied, raw logs must be parsed and transformed into usable representations. Template-based approaches such as Drain organize log messages into event templates and enable downstream sequence or statistical modeling [7]. DeepLog and related approaches model log sequences with deep learning to detect deviations from normal execution patterns [8]. These methods are powerful for homogeneous sequential logs, but applying the same sequence model indiscriminately to all enterprise sources is technically and economically difficult: Windows events, Linux authentication lines, SIEM exports, cloud JSON records and network flows do not share a single event grammar, sequence length, sampling rhythm or feature contract. A monolithic deep sequence model would therefore require costly global preprocessing, padding or template alignment, and would still risk learning dominant families while weakening minority

sources. Logminer takes a different route: it first normalizes enough information for audit and routing, then selects a compatible family-specific branch. Lightweight parsing is used in this first version, while template-based parsing is retained as a natural extension for Linux, HDFS and BGL-style logs.

C. Agent-Based Security Monitoring

Agent-based security monitoring has historical roots in distributed intrusion detection. AAFID proposed an architecture using autonomous agents to collect and analyze host-level events before aggregation [9]. Logminer follows this lineage in a more modest way: agents are software modules and services with clear responsibilities and exchange contracts. The goal is not full cognitive autonomy, but controlled specialization, modular replacement and future deployability.

III. METHODOLOGY

A. Architectural Scope

The proposed architecture follows a log processing chain from raw sources to analyst-readable incidents. Fig. 1 summarizes the logical workflow. Each agent has a restricted responsibility: the collector retrieves or receives raw logs; the parser detects formats and extracts fields; the normalizer maps extracted fields to a common schema; the router assigns events to a log family; the detector applies a family-specialized model; the correlator groups anomaly candidates into incidents; and the dashboard exposes results for human validation and audit.

The distribution validated in this work is primarily logical and modular. The service-level implementation is partially supported through FastAPI endpoints, a local JSONL bus and an optional Redis-based evolution. A large-scale multi-machine deployment is not claimed as a completed result.

B. Agent Roles and Interaction

Table I summarizes the main agents. This decomposition is important because it prevents the detector from becoming an opaque monolithic block. It also allows future replacement of one agent without redesigning the entire pipeline.

Communication is implemented through file-based artifacts, JSONL messages, FastAPI endpoints and optional Redis evolution. Coordination is handled by an orchestrated workflow, including discovered-source execution and route-based processing.

C. Normalized Event Contract

The common schema contains fields such as timestamp, severity, event identifier, source, host, user, source and destination IP, ports, category, subcategory and message. This common representation does not erase family-specific information; it provides the minimal interface required for model routing, scoring, correlation and visualization. The design follows log management principles in which normalization and retention of raw context are both necessary for auditability [10], [11].

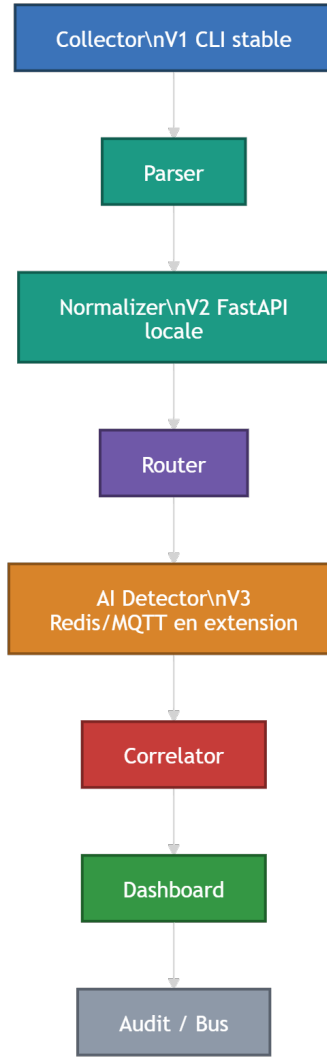


Fig. 1. Logical architecture of Logminer. The contribution is the controlled path from heterogeneous inputs to family-aware model selection and analyst-readable incident candidates, not a new standalone detector.

IV. FAMILY-AWARE ROUTING MECHANISM

B. Scientific Claim

A. Motivation

The central hypothesis of this work is that log families exhibit sufficiently different structures and distributions to justify specialized detection models. Windows event logs, Linux/auth logs, Wazuh alerts, HDFS traces, BGL HPC logs and network-flow datasets do not expose the same fields, semantics or anomaly patterns. A single global model may therefore learn dominant distributions from one family and behave poorly on another.

Logminer addresses this issue through family-aware routing. After normalization, each batch or stream is associated with a log family. The router selects a model compatible with that family. The detector then produces anomaly candidates in a unified output format, allowing downstream correlation and dashboard analysis.

The novelty is not the isolated use of parsing, machine learning or a dashboard. The novelty is an auditable specialization layer: heterogeneous logs are normalized, routed according to family, scored by compatible lightweight models and correlated into incidents while preserving the route evidence that led to model selection. This strategy aims to improve robustness and analyst usability in the presence of heterogeneous formats and distributions; this hypothesis must be evaluated experimentally against monolithic baselines. The claim is therefore architectural and experimental rather than algorithmically theoretical. A negative or neutral result on one dataset does not invalidate the claim if the architecture still exposes route reasons, fallback behavior and feature/model compatibility boundaries.

TABLE I
SPECIALIZED AGENTS IN THE LOGMINER ARCHITECTURE

Agent	Role	Output
Collector	Retrieves local or exported logs from Windows, Linux, SIEM or dataset folders	Raw files and manifests
Parser	Detects format and extracts timestamp, host, event, severity, source and message fields	Structured events
Normalizer	Maps heterogeneous fields to a common event schema	Normalized records
Router	Identifies the family of each log batch or event stream	Model selection decision
Detector	Applies a lightweight supervised or non-supervised model specialized by family	Anomaly candidates
Correlator	Groups anomalies by time, source, host, severity or context	Interpretable incidents
Dashboard/Audit	Displays events, anomalies, incidents and analyst decisions	Visual analysis and traceability
Supervisor	Observes sources/resources and adapts workflow parameters	Policy-based bounded workflow control

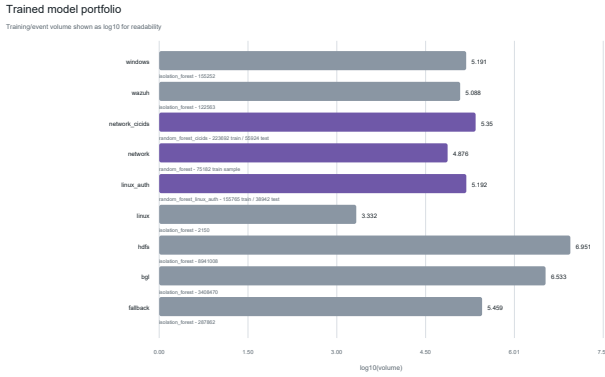


Fig. 2. Model portfolio used by Logminer across Windows, Wazuh, Linux/auth, network, HDFS, BGL and fallback families. Each artifact is selected according to routing confidence and family compatibility.

C. Router Formalization

Let x be a normalized event batch, $\mathcal{F}$ the set of supported log families, and M_f the detector associated with family $f \in \mathcal{F}$. The router computes a compatibility score:

$$S(f, x) = w_n N(f, x) + w_c C(f, x) + w_m M(f, x), \quad (1)$$

where $N(f, x)$ measures filename or source-name evidence, $C(f, x)$ measures the presence of expected normalized or native columns, and $M(f, x)$ measures marker tokens observed in sampled values. The selected family is:

$$f^* = \arg \max_{f \in \mathcal{F}} S(f, x), \quad (2)$$

with confidence:

$$\gamma(x) = \frac{S(f^*, x)}{\sum_{f \in \mathcal{F}} S(f, x) + \epsilon}. \quad (3)$$

If $\gamma(x) < \tau$ or if mandatory evidence is missing, the router selects the fallback detector. The detector output is then represented as a unified anomaly candidate:

$$a = \langle t, h, f^*, m, y, q, r \rangle, \quad (4)$$

where t is the timestamp, h the host or source, m the selected model, y the predicted candidate label, q the anomaly score or confidence, and r the routing reason. This formalization makes the routing decision auditable without claiming a learned routing policy.

The weights w_n , w_c and w_m are fixed policy parameters in the current prototype. They are set empirically to favor high-confidence structural evidence before weaker lexical cues: explicit family columns and dataset-specific fields receive the strongest contribution, marker tokens provide supporting evidence, and source-name evidence is used mainly as a tie-breaker or early hint. The implementation also uses priority ordering and fallback thresholds to avoid brittle decisions when evidence is sparse. Table II reports the exact additive constants used in the current prototype. In practice, routing decisions are not expected to be highly sensitive to small perturbations such as changing the generic column weight from +8 to +10, because most supported families are separated by multiple simultaneous signals and by fallback priority rules rather than by a single marginal feature.

The different magnitudes in Table II should be read as rule-specific additive evidence rather than as globally comparable learned coefficients. Heavy constants are attached to highly discriminative markers or to fallback-only conditions, and they are applied only when the corresponding evidence is present for the candidate family. A sparse family such as Linux/auth is therefore not penalized merely because another family has a larger possible maximum score; unsupported or absent descriptors do not contribute to competing families. The confidence $\gamma(x)$ is used as an audit and fallback signal together with mandatory evidence checks, not as the only routing criterion. A softmax or MinMax normalization could make scores numerically comparable, but it could also inflate weak noisy evidence when all families have low support. For this first prototype, explicit evidence thresholds and fallback routing were preferred for interpretability; calibrated score normalization remains future work.

A full sensitivity analysis over these weights is not claimed in this paper; it is included in future work together with the factorial ablation of routing, feature spaces and model choice.

D. Ablation Protocol

To make the claim testable, the experimental protocol includes an ablation study. Table III defines the comparisons. Two levels are reported. First, a controlled common-space ablation forces both the global model and the family-aware models to use the same minimal feature representation. Second, an operational ablation compares the same global common-feature baseline with the full specialized configurations, where each family can use its native feature space and trained detector. A complete factorial isolation of routing,

TABLE II
NUMERICAL ROUTING POLICY CONSTANTS USED IN THE PROTOTYPE

Evidence	Weight	Use in current router
Matched family column	+8 each	Generic column evidence for supported families
Generic family marker	+35	Marker token found in sampled values or dataset fields
Family-specific feature hit	+10 to +12 each	Windows, Linux, HDFS, BGL and network evidence bundles
Dense network profile	+20	Numeric tabular profile typical of network-flow data
Filename hint	+45	CICIDS or Linux/auth source-name evidence
Specialized dataset bundle	+70 to +120	Linux/auth, Wazuh and normalized SIEM evidence
Known unsupported subtype	+140 fallback	Apache, CEF/LEEF, CloudTrail or unknown subtype
Corrupted input marker	+120 fallback	Corrupt, incomplete or broken input indicators
Syslog subtype	+140 linux, -40 windows	Prevents syslog batches from being routed to Windows

TABLE III
ABLATION PROTOCOL USED TO EVALUATE FAMILY-AWARE SPECIALIZATION

Configuration	Purpose
Global common model	Main baseline: one detector applied to all families using a minimal shared feature space
Family-aware common models	Controlled test: family selection and per-family detectors under the same minimal shared features
Family-aware native models	Operational test: routing plus family-compatible native features and specialized detectors
Without correlation	Measures detector-level candidate production before incident grouping
With correlation	Measures the complete pipeline from anomaly candidate to analyst-readable incident
Factorial routing/features/models	Future Work: planned extension to isolate the independent causal effect of each component

features and model family is identified as future work; the present evaluation separates a constrained routing effect from the complete specialization package.

Table IV maps the main hypotheses to the level of evidence available in this paper. This table is included to make the contribution falsifiable and to separate demonstrated behavior from future work.

E. Fallback Routing

The fallback model is selected when the router cannot assign a source to a known family with sufficient evidence from file type, column names, marker tokens or sampled values. It is also selected for records explicitly normalized as unknown or marked as corrupted/incomplete. It is not intended to maximize detection performance. Its role is graceful degradation: unknown formats still produce low-confidence anomaly candidates instead of silently breaking the pipeline. Fallback outputs therefore require correlation or analyst validation before interpretation.

TABLE IV
HYPOTHESES AND EVIDENCE BOUNDARIES

Hypothesis	Expected behavior	Evidence boundary
H1	A monolithic common model can remain competitive when all families use the same minimal features	Confirmed by the controlled ablation; routing alone is not claimed as universally superior
H2	Family-aware routing helps when it enables native feature spaces and compatible detectors	Supported for Linux/auth and UNSW in the operational ablation; CICIDS is already near saturation
H3	Sequential system logs require sequence-aware branches	Supported as a limitation by HDFS false positives; Drain-like and temporal detectors remain future work
H4	Fallback routing prevents unknown or corrupted inputs from breaking the workflow	Supported as graceful degradation; fallback outputs are not confirmed intrusions
H5	The prototype is feasible on a local workstation or gateway	Supported by local latency and memory results; SOC-scale throughput is not claimed

TABLE V
FALLBACK BEHAVIOR ON CORRUPTED INPUT

Input	Rows kept	Candidates	Incidents
corrupt_incomplete.log	2	0	0

V. IMPLEMENTATION

The prototype is implemented in Python with a modular source tree. The stable version uses command-line execution, CSV artifacts and local folders. The service-oriented version exposes core workflow actions through FastAPI, including source discovery, detection and correlation. A local JSONL bus supports message-oriented traces, while Redis is documented as an optional extension.

The current local buffering strategy is intentionally simple. Collectors and parsers persist raw files, manifests, normalized records and bus messages as local file artifacts before detection. The detector then processes bounded samples or prepared batches rather than keeping an unbounded in-memory queue. If detection lags behind collection, the backlog remains visible in files and JSONL traces, but the prototype does not yet implement production-grade queue admission, back-pressure, retry policies or priority scheduling. This design is acceptable for a local analytical prototype, but it is not sufficient for sustained SOC ingestion under bursty or adversarial traffic; Kafka, MQTT, Redis Streams or an equivalent broker would be needed for durable asynchronous scaling.

The detector portfolio includes supervised Random Forest models for labeled families such as Linux/auth, CICIDS2017 and UNSW/CIC-DDoS, as well as non-supervised Isolation Forest or ensemble-style detectors for sources where labels are incomplete or absent. Non-supervised outputs are explicitly called anomaly candidates.

A web dashboard supports exploration of anomalies, incidents, counters, filters, timeline-style views and analyst decisions. This dashboard is an exploitation layer: it does not prove the anomaly by itself, but helps analysts validate, reject

TABLE VI
POLICY-BASED SUPERVISORAGENT CAMPAIGN

Indicator	Value
Completed cycles	30/30
Distinct selected sources	5
Routed families	fallback, linux
Mean cycle duration	6.3164 s
Maximum cycle duration	11.1918 s
Window range	15–20 min
Sample range	300–1000 rows
Audit trace recorded	yes

or reinterpret candidate incidents.

A. Correlation Logic

The correlator is intentionally explainable rather than opaque. It groups anomaly candidates by a configurable temporal window, host, user, source, category, subcategory, protocol and destination port. Incident priority combines event volume, maximum severity, diversity of event identifiers, security category weight and anomaly-score depth. For a candidate set A , the correlator emits an incident I when candidates share a context key and fall within the window Δt . The priority score is a weighted combination of candidate count, severity, source diversity and anomaly-score depth. This correlation step improves analyst readability, but it is treated as a pipeline component rather than as the main algorithmic contribution of the paper. The present evaluation reports the resulting incidents and timing overhead; it does not prove that every merged incident is analytically correct. A dedicated correlation-quality evaluation with analyst labels, window-size sensitivity and incorrect-merge analysis remains future work.

B. Policy-Based Workflow Orchestration and Control

The current implementation uses software agents with specialized roles, explicit message exchange and workflow coordination. To move beyond a purely static script, a bounded SupervisorAgent was added. It performs a perception-state-decision-action cycle: it observes available log candidates and local CPU/RAM state, builds a persistent memory, chooses a source and workflow parameters, then launches parsing, routing, detection, correlation and audit. Its policy favors unexplored sources, reduces sampling under uncertainty or repeated errors, increases correlation windows when fallback routing becomes frequent, and records each decision in audit and bus traces. The current decisions are policy-based and explainable rather than learned; therefore, the supervisor is evaluated as bounded workflow control, not as adaptive artificial intelligence. Table VI summarizes a 30-cycle SupervisorAgent campaign.

In this campaign, the supervisor first processed five different sources before repetition, routed unsupported web/SIEM/cloud formats to fallback, routed syslog to the Linux detector, and reduced sampling only for corrupted unknown input. After three fallback routes, it increased the correlation window from 15 to 20 minutes to favor broader analyst review; the value then remained bounded at 20 minutes during the rest

of the campaign. In the current implementation, adaptation is bounded: each decision recomputes parameters from fixed defaults, the correlation window is capped by policy at 20 minutes for repeated fallback routes and 30 minutes for sequential HDFS/BGL-style sources, and sampling is reduced to a lower bounded profile rather than repeatedly divided. Therefore, the present policy does not accumulate an unbounded window increase across cycles.

Under prolonged stress, for example a multi-hour stream of corrupted logs, this bounded behavior protects resources but does not solve the observability problem by itself. Repeated fallback routing, maximal windows and minimal sampling would be visible in the audit trail, but they could still create a blind spot if treated as normal operation. The current policy should therefore be interpreted as graceful degradation rather than attack-resistant control. The current telemetry records each decision, reason, selected source, routed family, sample size, window size and status in JSONL bus messages and audit traces, which makes ping-pong behavior between conservative and normal sampling profiles traceable and manually interruptible through the workflow/API control layer. No such sampling oscillation was observed in the 30-cycle campaign. A stricter future version should add an automatic oscillation watchdog, fallback-rate alerts, bounded degradation budgets, source quarantine rules, analyst notification and operation over a distributed message bus with back-pressure.

VI. EXPERIMENTAL SETUP

Table VII lists the families used in the initial evaluation. The selection mixes local logs, SIEM-style exports, public system logs and public network datasets.

A. Evaluation Protocol and Experimental Rigor

The evaluation follows a validation-based protocol adapted to the heterogeneous nature of the sources. For labeled supervised datasets, metrics are reported on held-out test partitions or predefined evaluation splits, using F1-score and false positive rates to capture both detection quality and analyst workload. For unlabeled operational exports, such as Wazuh or local Windows logs, outputs are explicitly interpreted as anomaly candidates and correlated incidents rather than confirmed attacks. Workflow latency and CPU/RAM snapshots are reported separately because they characterize deployment feasibility rather than detection accuracy.

The routing ablation is split into two complementary views. The controlled common-space comparison constrains all families to the same minimal representation, whereas the operational comparison evaluates the full specialization package composed of family identification, native feature compatibility and specialized model selection. This protocol is designed to reduce overclaiming: the controlled ablation tests whether routing alone dominates under shared features, while the operational ablation tests whether routing enables compatible feature spaces and models. All results should therefore be read as evidence for a lightweight family-aware analytical

TABLE VII
DATASETS AND SCENARIOS USED IN THE INITIAL EVALUATION

Category	Source	Nature	Objective
Local real	Windows	Windows system logs	Test EVT/X/ML collection, parsing and local candidate detection
Real/SIEM	Event/Application/System/Security Wazuh exports	SIEM/HIDS events and alerts	Test non-supervised detection on operational security signals
Real tabular	Linux/auth	Labeled authentication logs	Supervised evaluation and false positive analysis
Public system	HDFS	Distributed-system logs with labels	Validate detectors on sequential system logs
Public system	BGL	BlueGene/L logs with labels	Validate detectors on HPC logs
Public network	CICIDS2017	Labeled network flows	Supervised detection of known attack classes
Public network	UNSW/CIC-DDoS	Network/DDoS flows	Chunk-based 80/20 evaluation
Robustness	Apache, CEF/LEEF, Cloud-Trail, Linux auth, incomplete log	Multi-format and corrupted input	Test parser tolerance and preservation of unknown records

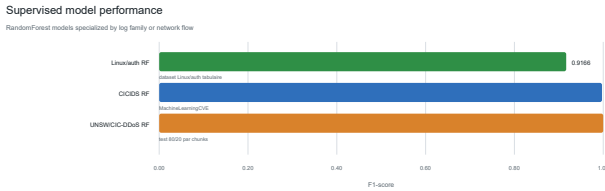


Fig. 3. F1-score of supervised Random Forest models on Linux/auth, CICIDS2017 and UNSW/CIC-DDoS families.

architecture, not as a universal benchmark victory over all monolithic or sequence-aware detectors.

The experimental artifacts were organized to support manuscript-level traceability: routed outputs, anomaly candidates, incidents, timing profiles, resource snapshots, figures and LaTeX tables are preserved as local files before being summarized in the paper. The present manuscript does not yet claim a public reproducibility package with fixed random seeds, containerized execution and model cards. Such an artifact release is planned before broader external replication.

VII. RESULTS AND DISCUSSION

A. Detection Results

Table VIII summarizes the principal results available in the current prototype. The supervised Random Forest models obtain strong F1 scores on CICIDS2017 and UNSW/CIC-DDoS, and a lower but still usable score on Linux/auth. HDFS remains more difficult with the current lightweight features, whereas BGL obtains high validation scores.

These results should not be interpreted as universal superiority. They show that the prototype can integrate several families and that family-specific models are operationally feasible. The routing effect is therefore isolated through the ablations below.

B. Routing Ablation

Table IX reports the controlled ablation. In this setting, both the global model and the family-aware models use the same minimal common feature space. This isolates routing under a constrained representation. The global model remains highly competitive and even slightly outperforms the per-family models on Linux/auth and UNSW in this specific

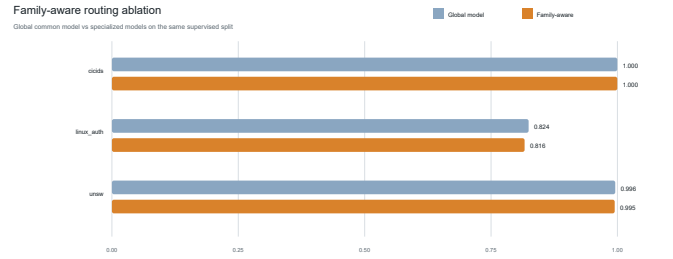


Fig. 4. Controlled common-space ablation. The figure compares global and family-aware models under the same minimal feature representation, so it should not be read as the full operational specialization result.

setting. This result is important: it prevents overclaiming and shows that routing alone does not guarantee higher F1 when all families are forced into the same representation.

Table X compares the same global common-feature baseline with the full specialized configurations. This second view better reflects the proposed contribution, because family-aware routing allows each family to use a compatible feature space and model. The full configuration improves Linux/auth and UNSW, while CICIDS remains slightly better under the global common baseline because that dataset is already nearly saturated.

These ablations refine the claim: family-aware routing should not be presented as a universal F1 improvement. Its value is controlled specialization and feature/model compatibility across heterogeneous families. In particular, it helps when a family benefits from a dedicated representation, as shown by Linux/auth and UNSW in the operational comparison.

C. False Positives

False positives matter because they directly affect analyst workload. Table XI reports representative false positive rates. HDFS shows a high false positive rate with the current lightweight row-level representation. This is a structural limitation for distributed-system logs, where event order and textual templates are central. The absence of temporal-window features is not a hard limitation of the normalizer schema, which can carry timestamps, event identifiers and

TABLE VIII
PRINCIPAL DETECTION RESULTS AVAILABLE IN THE PROTOTYPE

Family	Dataset	Model	Main result
Windows	Security/Application/System	Isolation Forest	Anomaly candidates and correlated incidents
Wazuh	January/October/December exports	Isolation Forest	122,563 events and 3,676 anomaly candidates
Linux/auth	Authentication logs	Random Forest	F1 = 0.916602
CICIDS	CICIDS2017 MachineLearningCVE	Random Forest	F1 = 0.997163
UNSW/CIC-DDoS	UNSW-NB15/CIC-DDoS network data	Random Forest	F1 = 0.999965
HDFS	Distributed-system logs	Ensemble / Isolation Forest	Best selection around F1 = 0.599333
BGL	BlueGene/L HPC logs	Ensemble / Autoencoder / Isolation Forest	Best selection around F1 = 0.994333

TABLE IX
CONTROLLED COMMON-SPACE ABLATION

Variant	Family	Rows	F1	FP/1000
Global common	CICIDS	3,000	0.999667	0.000
Family-aware	CICIDS	3,000	0.999667	0.000
Global common	Linux/auth	3,000	0.824368	50.000
Family-aware	Linux/auth	3,000	0.816399	53.333
Global common	UNSW	3,000	0.995662	1.667
Family-aware	UNSW	3,000	0.994660	2.000

TABLE X
OPERATIONAL ABLATION: GLOBAL COMMON BASELINE VS FULL SPECIALIZED CONFIGURATIONS

Family	Global F1	Family-aware F1	Δ F1
Linux/auth	0.824368	0.916602	+0.092234
CICIDS	0.999667	0.997163	-0.002503
UNSW	0.995662	0.999965	+0.004303

raw messages. It is a deliberate lightweight design constraint of this first prototype. Therefore, the current HDFS result should not be compared favorably against sequence-aware methods such as DeepLog or LogAnomaly. For HDFS/BGL-like sources, Logminer should be extended with a semantic parsing agent, for example Drain-like templates, and temporal windows before detection. The modular event contract is precisely intended to support this downstream algorithmic heterogeneity: HDFS/BGL batches could be routed to a Drain-like parser and an LSTM-style sequence detector without changing simpler tabular branches such as Linux/auth, CICIDS2017 or UNSW/CIC-DDoS.

D. Robustness and Workflow Performance

The parser and normalizer were tested on Apache, CEF/LEEF, CloudTrail, Linux/auth and incomplete logs. The objective is not to claim full universal parsing, but to verify that unexpected or corrupted lines are preserved as unknown records without interrupting the workflow.

The FastAPI quasi-real-time workflow was executed for five cycles with 8,537 lines per cycle. The mean workflow latency was 10.6473 seconds, with a minimum of 3.0865 seconds and a maximum of 19.8163 seconds. A 30-cycle resource campaign reported a mean workflow duration of 9.3300 seconds and maximum duration of 21.6007 seconds.

TABLE XI
REPRESENTATIVE FALSE POSITIVE RESULTS

Dataset	Model	Test lines	FPR	FP/1000
Linux/auth	Random Forest	38,942	0.047250	29.120
CICIDS2017	Random Forest	55,924	0.001933	1.037
UNSW/CIC-DDoS	Random Forest	1,474,193	0.001709	0.001
BGL	Ensemble selected	6,000	0.005667	2.833
HDFS	Ensemble selected	6,000	0.400667	200.333
Simulated Windows	Isolation Forest	6,000	0.000000	0.000

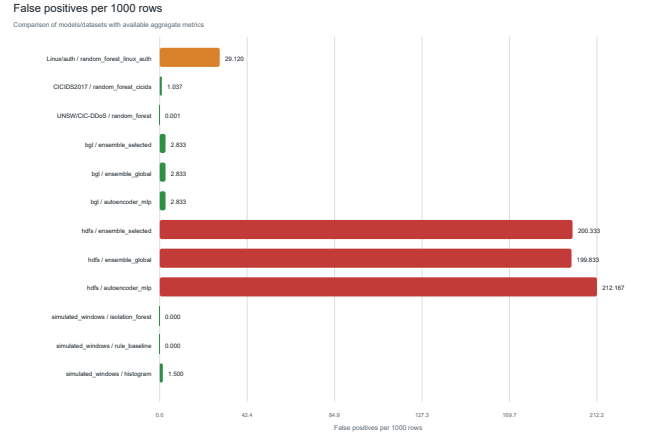


Fig. 5. False positive rates observed by log family and model. HDFS highlights the limitation of lightweight row-level features for sequence-oriented distributed-system logs.

The API/orchestrator process averaged 187.82 MB RAM and reached 221.20 MB, while the Logminer process averaged 495.41 MB and reached 655.39 MB. This memory footprint is lightweight compared with a full ELK-style stack, but it is still significant for direct installation on production endpoints. The intended deployment for this version is therefore a local analysis workstation or intermediate collection gateway; endpoint hosts may run lightweight collectors or export logs, while model inference and correlation are centralized. Routing itself is based on a bounded sample and a fixed number of family scores; its approximate cost is $O(k \times c \times s)$, where k is the number of families, c the inspected columns and s the sampled rows. A direct profiling run on 8,537 normalized Windows events shows that the router accounts

Multi-format parsing robustness
Check on Apache, CEF/LEEF, CloudTrail, Linux auth and incomplete log

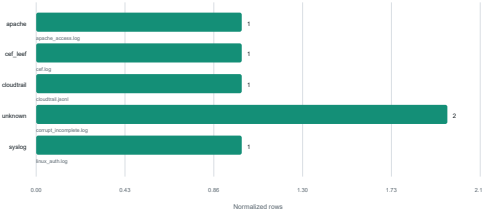


Fig. 6. Robustness check on multi-format inputs. Unknown or incomplete entries are preserved instead of breaking the pipeline.

TABLE XII

DIRECT TIMING PROFILE ON 8,537 NORMALIZED WINDOWS EVENTS

Step	Seconds	Share
Router	0.0449	0.9%
Detector	4.3460	91.3%
Correlator	0.3678	7.7%

for 0.9% of the measured routing-detection-correlation time, while model inference accounts for 91.3% and correlation for 7.7% (Table XII). These measurements support feasibility on a local machine or gateway, but do not establish industrial SOC throughput.

VIII. GENERAL DISCUSSION AND LIMITATIONS

A. Comparison with Standard Tools

A common reviewer objection is that ELK, OSSEC, Wazuh or fail2ban already process logs. This paper does not dispute their operational value. ELK-like stacks mainly provide ingestion, indexing, search, storage and visualization. Wazuh and OSSEC provide mature HIDS/SIEM functions based on agents, decoders and rules. fail2ban provides direct operational reaction to repeated authentication failures. Logminer is different in scope: it focuses on analytical specialization, dynamic model selection by log family, unification of supervised and non-supervised outputs as anomaly candidates, and correlation into analyst-readable incidents.

B. Scope and Limitations

Table XIV states the defensible scope of the prototype. This scope is necessary to avoid overclaiming.

The high HDFS false positive rate is an important example of this scope boundary. It does not invalidate the family-aware architecture, but it shows that sequential distributed-system logs require a different downstream detector family than lightweight row-level scoring. Because Logminer separates normalization, routing and detection, a future HDFS/BGL branch can route normalized events toward a Drain-like template parser followed by sequence-aware detection, while simpler tabular branches such as Linux/auth, CICIDS2017 and UNSW/CIC-DDoS continue to use lighter supervised models. This modularity is central to the proposed architecture: heavier algorithms can be attached only where the log family requires them.

TABLE XIII
QUALITATIVE POSITIONING AGAINST STANDARD TOOLS

Criterion	Logminer	fail2ban	OSSEC/Wazuh
Detection	Statistical/AI anomalies and contextual correlation	Rules on authentication failures and IP banning	Rules, decoding and HIDS/SIEM alerts
Sources	Windows, Linux, Wazuh, HDFS, BGL, network, cloud	Mainly exposed services and auth logs	Broad source coverage through agents and decoders
Adaptation	Multi-model routing by family	Manual filters and thresholds	Rules, lists and SOC configuration
Positioning	Lightweight analytical complement	Immediate operational protection	Mature production HIDS/SIEM base

TABLE XIV

SCOPE AND LIMITATIONS OF THE PROTOTYPE

Point	Defensible position
Software agents	Specialized cooperating software modules plus bounded supervisor control, not fully cognitive autonomous agents
Distribution	Logical and local service distribution; multi-machine deployment remains future work
Adaptive AI	Policy-based routing and workflow parameter adaptation, not continuous self-learning or optimized planning
Real time	Local quasi-real-time workflow, not strict industrial SOC real time
Asynchronism	Current FastAPI/JSONL workflow is mostly synchronous; Kafka, MQTT or Redis Streams would be required for back-pressure and worker parallelism
Deployment	Current prototype targets an analysis workstation or intermediate gateway, not a heavy detector installed on every endpoint
Tool comparison	Analytical complement to Wazuh/fail2ban/OSSEC, not replacement
Unsupervised results	Anomaly candidates, not confirmed intrusions

C. Threats to Validity

The main internal validity threat is that results combine multiple datasets, feature representations and model families. The controlled ablation mitigates this risk, but also shows that routing alone is not always superior under a common minimal feature space. The operational ablation evaluates routing, specialized feature spaces and specialized detectors together; it does not isolate the independent contribution of each factor. Therefore, the claim is limited to controlled specialization and feature/model compatibility, not universal dominance over monolithic classifiers. A stricter factorial design would require at least three independent axes: router choice, feature representation and detector family.

Statistical validity is limited because repeated-seed confidence intervals and routing-weight sensitivity sweeps are not yet reported. The current router constants are auditable policy parameters, but they are not calibrated probabilities. Dataset validity is also limited: very high F1 scores on network datasets require careful interpretation because class imbalance, duplicate flows, temporal splitting and train-test leakage can distort intrusion-detection benchmarks. HDFS demonstrates an explicit negative boundary of the current lightweight row-level representation and should be compared against sequence-aware parsers and detectors before any stronger claim is made.

for distributed-system logs.

External validity is limited by the selected datasets and local execution environment. Operational validity is limited because non-supervised anomalies require analyst confirmation, correlation quality is not yet evaluated against analyst-labeled incidents, and a real SOC deployment would require stronger ingestion guarantees, access control, retention policies, back-pressure handling, response workflows and stress tests over longer campaigns.

IX. CONCLUSION AND FUTURE WORK

This paper presented Logminer, a lightweight auditable family-aware routing architecture for heterogeneous log anomaly detection. Its main contribution is a specialization layer that makes model selection explicit, preserves routing reasons and fallback behavior, and connects common normalization with compatible family-specific detectors and interpretable correlation. Results across Windows, Wazuh, Linux/auth, HDFS, BGL, CICIDS2017 and UNSW/CIC-DDoS support the feasibility of the approach on a local infrastructure.

The ablation study refines the contribution: the family-aware strategy should be understood as controlled specialization rather than a guarantee of better F1 on every source. Further work includes a factorial ablation separating routing, features and model choice; stronger template-based parsing; routing HDFS/BGL-style logs toward a Drain-like semantic parsing agent followed by temporal or LSTM-style sequence detection; multi-machine deployment; active learning from analyst decisions; learned supervisor policies; longer supervisor stability campaigns; richer incident explanations; and integration with mature SIEM/HIDS platforms.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES

The authors used AI-assisted writing tools to improve language clarity, structure and readability during manuscript

preparation. The authors reviewed, edited and validated the technical content and take full responsibility for the final manuscript.

REFERENCES

- [1] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987, doi:10.1109/TSE.1987.232894. Available: <https://www.cs.colostate.edu/~cs656/reading/ieee-se-13-2.pdf>.
- [2] S. Axelsson, "Intrusion detection systems: A survey and taxonomy," Technical Report, Chalmers University of Technology, 2000. Available: <https://www.cse.msu.edu/~cse960/Papers/security/axelsson00intrusion.pdf>.
- [3] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316, doi:10.1109/SP.2010.25.
- [4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009, doi:10.1145/1541880.1541882.
- [5] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys and Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016, doi:10.1109/COMST.2015.2494502.
- [6] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 20, 2019, doi:10.1186/s42400-019-0038-7.
- [7] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *IEEE International Conference on Web Services*, 2017, pp. 33–40, doi:10.1109/ICWS.2017.13. Available: https://jiemingzhu.github.io/pub/pjhe_icws2017.pdf.
- [8] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *ACM CCS*, 2017, pp. 1285–1298, doi:10.1145/3133956.3134015.
- [9] J. S. Balasubramanian, J. O. Garcia-Fernandez, D. Isacoff, E. Spafford, and D. Zamboni, "An architecture for intrusion detection using autonomous agents," in *Annual Computer Security Applications Conference*, 1998, pp. 13–24. Available: <https://zzamboni.org/files/pubs/aafid-acsc98.pdf>.
- [10] National Institute of Standards and Technology, "Guide to computer security log management," NIST Special Publication 800-92, 2006, doi:10.6028/NIST.SP.800-92.
- [11] R. Gerhards, "The Syslog Protocol," RFC 5424, 2009, doi:10.17487/RFC5424.